Efficient MPI Circle Rendering with Adaptive Heuristic Decomposition Methods

An-Che Liang, Guan-Ming Chiu, Pei-Yu Wu
National Taiwan University
Team 12

1 Environment

The experiments were conducted on Ubuntu 24.04.1 LTS running Linux kernel 6.8.0-100-generic. The host system used an Intel Xeon Platinum 8352V processor at 2.10 GHz with a topology of 2 sockets, 36 cores per socket, 2 threads per core, and 2 NUMA nodes. The MPI runtime was Open MPI 4.1.6, and the compiler was `mpicc` backed by GCC 13.3.0.

The experimental configuration used for benchmarking is summarized as follows: We measured performance using MPI process counts of 1, 4, 8, 16, and 64; single-node experiments used up to 16 processes, while the 64-process runs were executed across four nodes with 16 processes per node (via the `hosts` file). Reported timings are the median of 10 runs and correspond to the total wall time printed by `renderer_mpi`.

2 Implementation

The final MPI renderer selects between two strategies at runtime according to the number of circles. Scenes with 500,000 circles or fewer use Mode A, named REPLICATE, whereas scenes above that threshold use Mode B, named PARTITION. The threshold was tuned empirically against benchmark data at 64 processes and can be overridden through the `RENDERER_THRESHOLD` environment variable.

2.1 Mode A: REPLICATE

In Mode A, rank 0 reads the CRDR file and broadcasts all circle records to every rank. Each rank owns a cyclic stripe of image rows, where rank r owns rows $r, r + P, r + 2P, \ldots$, with P denoting the amount of processes. Before rasterization, each rank prefilters the full record list to retain only circles whose bounding-box vertical range intersects its owned rows. The prefiltered list caches the bounding box and the first owned row, denoted y_{first}, so the inner raster loop remains branch-free with respect to ownership. Each rank rasterizes into a compact $W \times \text{local\_rows}$ RGB buffer, and `MPI_Gatherv` collects the compact slices before rank 0 reinterleaves them into the final image.

Cyclic row ownership balances work even when circles cluster in a vertical region, which a contiguous block assignment cannot guarantee. The compact per-rank buffer also avoids allocating a full $W \times H$ image on every process, which becomes increasingly important as P grows.

2.2 Mode B: PARTITION

In Mode B, rank 0 reads and broadcasts the full record array. A scatter-based distribution was measured to be approximately three times slower than broadcast on this cluster for equivalent payloads, so broadcast is used in both modes. Each rank then slices the record array locally: rank r owns a contiguous chunk of $count/P$ consecutive records in file order, with rank 0 holding the earliest, back-most circles and rank $P - 1$ holding the latest, front-most circles. Each rank rasterizes its

own records into a full $W \times H$ buffer of `uint32_t`-encoded pixels, where each pixel is packed as $((\text{rank}+1) \ll 24) \mid R \ll 16 \mid G \ll 8 \mid B$. Untouched pixels remain zero. A collective `MPI_Reduce` with `MPI_MAX` across all ranks then selects the correct color per pixel, because a larger rank marker indicates that a more forward circle, having larger rank, should win under the back-to-front operation. Using the predefined `MPI_MAX` operation on `MPI_UINT32_T` keeps the reduction on the hardware-accelerated collective path.

This mode reduces the per-rank rasterization cost to approximately $1/P$ of the serial work, at the cost of a full-image $W \times H \times 4$-byte buffer on each rank and a reduction across all pixels.

2.3 Other optimizations

The implementation applies per-row analytic x-range trimming with `sqrtf`, followed by the exact circle-inside test, to avoid iterating over pixels that are clearly outside the circle. The renderer writes opaque RGB values directly, which matches the assignment format in which records contain only `uint8_t` red, green, and blue components. The source also uses `#pragma GCC optimize("O3", "unroll-loops")` because the spec's compiling command does not specify an explicit optimization flag.

3 Test Cases

The metadata shows that the large cases use a 640×480 image with 1M, 2M, and 4M circles. Table 1 summarizes the benchmark inputs. The special `imbalance_c100000` case is smaller, using a 320×240 image with 100K circles, but it has much larger radius variation. Its maximum radius is approximately 1998 pixels, compared with roughly 20 pixels for the regular large cases. This makes the case suitable for stressing load balance.

Testcase	Circles	Image	Mean radius	Max radius
`imbalance_c100000`	100,000	320×240	25.62	1998.10
`medium_c200000`	200,000	640×480	10.49	20.00
`large_c1000000`	1,000,000	640×480	10.50	20.00
`large_c2000000`	2,000,000	640×480	10.50	20.00
`large_c4000000`	4,000,000	640×480	10.50	20.00

Table 1: Summary of benchmark test cases.

4 Benchmark Results

The benchmark uses the median wall time across 10 repetitions for each testcase and process-count pair, with the aggregate results listed in Table 2. The runtime, speedup, and efficiency trends are plotted in Figure 1, Figure 2, and Figure 3. The one-process MPI run serves as the serial baseline for speedup and efficiency because it exercises the same optimized renderer with $P = 1$.

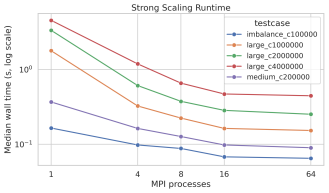

Figure 1: Strong-scaling runtime across process counts.

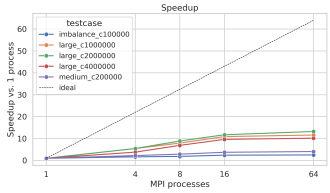

Figure 2: Speedup across process counts.

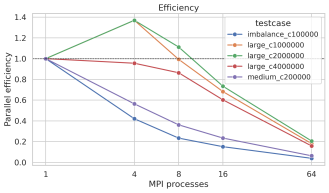

Figure 3: Parallel efficiency across process counts.

The best 64-process strong-scaling result is `large_c2000000`, which improves from 3.336 s on one process to 0.252 s on 64 processes, corresponding to a 13.24× speedup. On a single local

Testcase	1 proc (s)	4 proc (s)	8 proc (s)	16 proc (s)	64 proc (s)	64-proc speedup	64-proc efficiency
`imbalance_c100000`	0.165	0.098	0.088	0.068	0.065	2.54×	4.0%
`medium_c200000`	0.368	0.163	0.127	0.098	0.090	4.09×	6.4%
`large_c1000000`	1.780	0.325	0.224	0.163	0.153	11.63×	18.2%
`large_c2000000`	3.336	0.609	0.375	0.284	0.252	13.24×	20.7%
`large_c4000000`	4.534	1.186	0.656	0.470	0.445	10.19×	15.9%

Table 2: Median runtime, speedup, and efficiency for each testcase.

node, the same case improves to 0.284 s on 16 processes, which is an 11.75× speedup with 73.4% efficiency. The smaller cases still improve, but their efficiency declines because fixed costs such as record broadcast, gather, PNG writing, MPI launch overhead, and multi-node communication occupy a larger share of total runtime.

5 Problem Size Per Process

At 64 processes, throughput increases with larger scenes, as shown in Figure 4: 1.54 million circles per second for `imbalance_c100000`, 2.22 million circles per second for `medium_c200000`, 6.54 million circles per second for `large_c1000000`, 7.94 million circles per second for `large_c2000000`, and 8.99 million circles per second for `large_c4000000`.

With 64 processes, the workload grows from 15,625 circles per process for `large_c1000000` to 62,500 circles per process for `large_c4000000`. Throughput improves from 6.54 to 8.99 million circles per second because fixed MPI and output costs are amortized over more circles, but runtime still rises from 0.153 s to 0.445 s. The larger cases are faster per circle, but they still take longer overall.

6 Load Balance

The renderer logs per-rank render time statistics and reports imbalance as $\max/\mathrm{mean}$. The measured imbalance values are plotted in Figure 5.

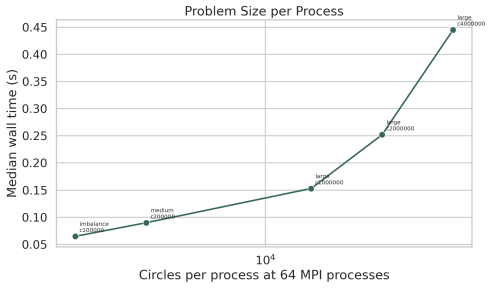

Figure 4: Throughput as a function of problem size at 64 processes.

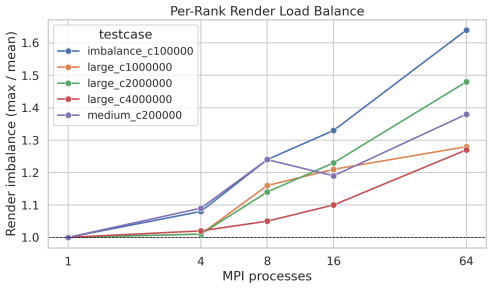

Figure 5: Per-rank render imbalance across benchmark cases.

The three large scenes, namely `large_c1000000`, `large_c2000000`, and `large_c4000000`, use Mode B. In this mode each rank renders a disjoint slice of records into a full image, so imbalance reflects variation in circle counts across ranks. Because records are split evenly, imbalance arises from radius variation rather than spatial clustering. At 16 processes the imbalance is 1.21, 1.23, and 1.10 respectively, and at 64 processes it is 1.28, 1.48, and 1.27. These values remain controlled because the large scenes use uniformly sized circles with a maximum radius of 20 pixels.

The two smaller scenes, `imbalance_c100000` and `medium_c200000`, use Mode A. In this mode each rank rasterizes prefiltered circles into its cyclic row stripe, so imbalance reflects whether very large circles disproportionately cover certain rows. At 64 processes, `medium_c200000` has imbalance 1.38, and `imbalance_c100000` has the highest imbalance at 1.64. The special imbalance case contains circles with radii up to 1998 pixels in a 320 × 240 image, so a single circle can span almost

the full image height. Cyclic row distribution spreads that coverage across all ranks rather than concentrating it in one contiguous block, which keeps imbalance below $2\times$ despite the extreme radius variation. Even with this imbalance, wall time improves from 0.165 s on one process to 0.065 s on 64 processes.

7 Complexity Analysis

Let N be the number of circles, W and H be the image width and height, P be the number of MPI processes, and A_i be the number of pixels covered by circle i after bounding-box trimming. Define $S = \sum_i A_i$. The serial renderer has work $O(N + S)$ and memory $O(WH)$, excluding the input record array.

Property	Type	Mode A: REPLICATE	Mode B: PARTITION
Input records per rank	Space	$O(N)$	$O(N)$
Raster work per rank	Time	$O(N + S/P)$	$O(N/P + S/P)$
Image memory per rank	Space	$O(WH/P)$	$O(WH)$
Final collective	Time	$O(WH)$	$O(PWH)$

Table 3: Asymptotic comparison of the two MPI rendering modes.

Table 3 summarizes the asymptotic tradeoffs, where T is the runtime threshold used to choose between modes. For both modes, every rank receives and scans all N circle records, so the broadcast and prefiltering cost is $O(N)$ per rank. In Mode A, rasterization is divided by cyclic image rows; under balanced coverage, each rank renders about $O(S/P)$ pixels, although very large or clustered circles can increase the slowest-rank cost. The compact local image buffer requires $O(WH/P)$ memory per rank, plus $O(N)$ input storage and prefiltered metadata. The final gather communicates $O(WH)$ total image data to rank 0.

In Mode B, each rank still receives the full input array, but rasterizes only $O(N/P)$ records. Its rasterization work is therefore $O(N/P+S/P)$ when circle areas are evenly distributed across record chunks. The tradeoff is memory and reduction cost: every rank keeps a full packed image buffer, so per-rank image memory is $O(WH)$, and the `MPI_Reduce` over pixels takes $O(WH)$ values from each of the P ranks, or $O(PWH)$ total element work before collective implementation constants.

The selection rule follows from this tradeoff. Mode A should be used for smaller inputs, $N \leq T$, because it avoids the full-image reduction and keeps per-rank image memory at $O(WH/P)$. Mode B should be used for larger inputs, $N > T$, because reducing raster work from $O(N)$ to $O(N/P)$ per rank outweighs the extra $O(WH)$ image memory per rank and the $O(PWH)$ final reduction.

8 Conclusion

The MPI renderer is correct on all benchmarked test cases and demonstrates meaningful strong scaling under the 64-process competition configuration. The adaptive strategy, which switches from cyclic row decomposition in Mode A for smaller scene sizes to record partitioning with a painter reduction in Mode B for larger scene sizes, provides good performance across both regimes. The best measured result is a $13.24\times$ speedup on `large_c2000000` under Mode B, while the 4M-circle case achieves the highest throughput at 8.99 million circles per second.

The main cost in Mode A is that every rank scans all records during prefiltering, so broadcast size and prefilter time grow with total circle count regardless of P. Mode B addresses this by giving each rank only $count/P$ records to rasterize, but it trades that improvement for a full-image `uint32_t` buffer per rank and a pixel-level `MPI_MAX` reduction. The threshold of 500,000 circles was chosen where the reduction overhead in Mode B becomes cheaper than the per-rank prefilter scan cost in Mode A at 64 processes.